Microservices Architecture Guide v3.5
Architecture Team

1. Service Boundaries

Services are organized around business capabilities, following Domain-Driven Design principles. Each service owns its data and exposes a well-defined API boundary. Services communicate through synchronous REST APIs or asynchronous event streams.

Current service catalog:
- user-service: Authentication, authorization, user management
- order-service: Order creation, modification, cancellation
- inventory-service: Stock management, reservations, warehouse sync
- payment-service: Payment processing, refunds, billing
- notification-service: Email, SMS, push notifications
- analytics-service: Event processing, reporting, dashboards

2. Service Communication

Synchronous communication (REST/gRPC) is used for request-response patterns where the caller needs an immediate answer. Timeout budgets are enforced: 500ms for user-facing calls, 5 seconds for internal calls.

Asynchronous communication (Kafka) is used for event-driven patterns, eventual consistency, and cross-service workflows. Events follow the cloud events specification with mandatory fields: id, source, type, timestamp.

Service mesh (Istio) handles mTLS, load balancing, circuit breaking, and retry policies. Observability is automatic through sidecar proxies.

3. Data Management

Each service owns its database (database-per-service pattern). Direct database access between services is strictly prohibited. Data consistency across services is achieved through:

- Saga pattern: Distributed transactions are choreographed through events
- Event sourcing: State changes are captured as immutable events
- CQRS: Read and write models are separated for performance

4. Resilience Patterns

Circuit breaker: Prevents cascade failures by stopping requests to failing services. The circuit opens after 5 consecutive failures and half-opens after 30 seconds.

Bulkhead isolation: Each downstream service call uses an independent thread pool. A slow service cannot exhaust resources needed for other calls.

Retry with backoff: Failed requests are retried with exponential backoff (1s, 2s, 4s) up to a maximum of 3 attempts. Jitter is added to prevent thundering herd.

Timeout budgets: Each service call has a defined timeout. Callers track remaining time and propagate shorter timeouts to downstream calls.